

Assignment-2

Introduction

This SimpleShell is implemented in C as per the assignment requirements. It supports:

- Reading user
- Parsing commands and arguments
- Executing commands using fork + exec
- Supporting multiple Unix commands and local executables
- Handling pipelines (|) between commands
- Providing a history feature (list of commands entered)
- On termination (Ctrl-C or "exit"), displaying details such as PID, start time, and duration for each executed command.

Approach

The implementation was divided into clear functional blocks:

- Command reading and parsing
- Process creation and execution using fork() and execvp()
- Pipe handling for commands with "|"
- Signal handling for Ctrl-C to terminate and show execution history
- In-memory history management to track commands executed in the session

This modular approach ensures clarity and makes the code easy to extend for additional features in the future.

AI usage

AI Code Snippets

AI assistance was used only in limited parts of the shell implementation.

- strtok() for tokenizing input and some help for parsing input
- strncpy() for copying command strings
- strlen(), strcmp(), strchr(), strcspn() for string operations
- gettimeofday() to record start and end time of commands
- time_t for epoch timestamps of command start

These are standard C library functions and POSIX calls just used to make code organized and well maintained.

Workflow

The workflow of SimpleShell is as follows:

1. Prompt: Display a prompt string ("Group_81_A2\$ ").
2. Read Input: Use `fgets()` to safely read user input from `stdin`.
3. Parse Command:
 - If input contains '|', split into multiple commands for pipeline execution.
 - Otherwise, split by whitespace into command + arguments.
4. Execution:
 - For single commands: fork and `execvp`.
 - For pipelines: fork multiple children, connect them with pipes using `dup2`.
5. History Management:
 - Each command executed is stored in in-memory history with PID, start time, and end time.
 - The "history" command shows a list of commands.
6. Termination:
 - On Ctrl-C or "exit", the shell prints a detailed execution summary with command string, PID, start time, and duration.

Supported Commands

All commands listed in the assignment specification have been tested and are supported by SimpleShell:

- `ls`
- `ls /home`
- `echo you should be aware of the plagiarism policy`
- `wc -l fib.c`
- `wc -c fib.c`
- `grep printf helloworld.c`
- `ls -R`
- `ls -l`
- `./fib 40`
- `./helloworld`
- `sort fib.c`
- `uniq file.txt`
- `cat fib.c | wc -l`
- `cat helloworld.c | grep print | wc -l`

The "fib" and "helloworld" are the executables of a Fibonacci number calculator and hello world programs, respectively (C code) that are available in the directory . File.txt is also attached.

Unsupported Commands / Limitations of Shell

Some commands or styles are not supported due to scope limitations:

- **Quoting / Escaping:**

Example: `echo "Hello World"`

Reason: Parsing only splits on whitespace.

- **I/O Redirection:**

Example: `ls > out.txt`

Reason: Assignment required only pipes, not redirection.

- **Background Execution:**

Example: `ls &`

Reason: Job control was not required in the assignment.

- **Advanced Shell Built-ins:**

Example: `cd`, `export`, `alias`

Reason: These modify the shell's environment, which is beyond our Shell's requirements.

- **Persistent History:**

Example: Restart shell and type history

Reason: The shell only stores commands during the current session; history is lost after termination.

- **Fixed Array Sizes:**

Example: very long commands with >100 arguments

Reason: The shell uses fixed-size buffers (e.g., `args[100]`), so extremely large inputs may be truncated or fail.

- **Limited Error Messages:**

Example: running `./not_exist`

Reason: Errors are displayed in a simple format (Execution failed) instead of detailed system-level messages.

Contributions

Somya	2024560	Implemented shell loop, parsing logic, single command execution, and history management. Integrated signal handling for Ctrl-C and cleanup.
Arjun Khatri	2024106	Implemented pipeline execution using fork, pipe, and dup2. Helped in debugging and testing supported commands.

References

- Lecture 06:
 1. Infinite shell loop with prompt
 2. Reading user input from standard input
 3. Splitting input into tokens (parsing)
- Lecture 07:
 1. fork() system call
 2. execvp() to execute commands
 3. waitpid() to wait for child process
 4. Error handling style with simple printf
- Lecture 08:
 1. pipe() system call
 2. dup2() to redirect input/output between processes
 3. sigaction() to handle Ctrl-C (SIGINT)

Github Link

[Github Link](#)